

- reporting diario de tienda
 - seguimiento de promociones
 - análisis de pricing
-

♦ **Venta neta real (post-devoluciones) → la importante para negocio**

Aquí sí:

Venta neta real = venta neta – devoluciones

Donde:

- devoluciones vienen de:
 - `returns.refund_amount`
 - o mejor aún: `return_items.line_refund_amount` (más granular)

👉 Esta es la que usamos para:

- performance real de tiendas
 - KPIs ejecutivos
 - comparativas entre periodos
-

⚠ **Decisión clave de modelado (esto lo quiero ver en tu gold)**

No mezcles esto “implícitamente”.

Espero:

- o bien dos métricas separadas
- o bien una métrica clara tipo:
 - `net_sales_before_returns`
 - `net_sales_after_returns`

Si lo metes todo en una sola sin trazabilidad → mal.

2) Granularidad de la tabla de hechos

Aquí voy a ser bastante claro:

necesitamos ambos niveles, pero con un propósito distinto.

♦ Nivel línea de ticket (OBLIGATORIO)

Grano:

1 fila = 1 `sale_item_id`

Porque:

- vendemos variantes (talla + color)
- el análisis real de negocio ocurre aquí

Esto te permite:

- unidades vendidas
- mix de producto
- descuentos por producto
- margen (si lo trabajas)
- cruce con devoluciones (`return_items`)

👉 Esta debería ser tu fact principal de ventas

♦ Nivel ticket (derivado / agregado)

Grano:

1 fila = 1 `sale_id`

Se usa para:

- ticket medio
- número de tickets
- análisis de comportamiento cliente

👉 No la usaría como única fact, sino como:

- agregado derivado
 - o tabla específica para BI
-

♦ Devoluciones

Aquí hay otro punto crítico:

Nunca modelos devoluciones solo a nivel cabecera (`returns`).

Necesitamos:

1 fila = 1 return_item_id

Porque:

- la devolución puede ser parcial
- afecta a productos concretos
- necesitas cruzarlo con la venta original (**sale_item_id**)

3) Impuestos (IVA)

Aquí hay una limitación importante del modelo:

👉 No hay ningún campo de impuestos en las tablas.

Ni en:

- sales
- sale_items

Interpretación correcta

Los importes:

- subtotal_amount
- total_amount
- line_total

👉 deben interpretarse como importes finales ya calculados (es decir, lo que el cliente paga)

Qué implica para gold

- **No esperes reconstruir IVA → no se puede**
- **Trabaja siempre con:**
 - importes tal cual vienen
- **Sé consistente:**
 - no intentes mezclar “con IVA” vs “sin IVA”

NUESTRA SOLUCION:

Hemos adaptado el modelo gold para reflejar exactamente vuestra definición de negocio. Ahora distinguimos de forma explícita venta neta comercial (`net_sales_before_returns`) y venta neta real post-devoluciones (`net_sales_after_returns`), evitando mezclar conceptos en una única métrica. Además, la fact principal de devoluciones se ha llevado a granularidad de línea (1 fila por `return_item_id`), permitiendo trazabilidad completa con la venta original por `sale_item_id`.

También mantenemos una vista derivada a nivel ticket (1 fila por `sale_id`) para análisis de ticket medio y comportamiento cliente, y en todo el reporting trabajamos con importes tal como vienen en origen (sin reconstrucción de IVA, ya que no existe ese dato en el modelo).

Cambios aplicados

- En `fct_sales` (`models/gold/facts/fct_sales.sql`) ahora quedan explícitas ambas métricas:
 - `net_sales_before_returns` (neta comercial: bruta - descuentos)
 - `net_sales_after_returns` (neta real: neta comercial - devoluciones)
- También se añadió columnas para trazabilidad:
 - `gross_amount` (importe a precio lista)
 - `discount_amount`
 - `amount_refunded`
- Creación de la fact principal de devoluciones a nivel línea:
 - `models/gold/facts/fct_return_items.sql`
 - grano: 1 fila = 1 `return_item_id` (como pedía el cliente)
 - enlaza con venta original por `sale_item_id`
- `fct_returns` se mantiene como fact derivada de cabecera (útil para BI), pero ahora alimentada desde `fct_return_items` y marcada como derivada:
 - `models/gold/facts/fct_returns.sql`
- Adición fact derivada a nivel ticket:
 - `models/gold/facts/fct_sales_tickets.sql`
 - grano: 1 fila = 1 `sale_id`
- Actualización el agregado diario:
 - `models/gold/aggregates/daily_store_performance.sql`
 - ahora expone:
 - `gross_sales_amount`
 - `net_sales_before_returns`
 - `refunded_amount`
 - `net_sales_after_returns`
 - `return_rate_amount`

2. Sobre el Producto y Stock (Dimensiones y Snapshots)

Jerarquía: "¿Un Producto puede tener varias Variantes (talla/color) con distintos precios, o el precio siempre es el mismo para todas las variantes de un modelo?"

Histórico (Snapshot): "¿Qué os interesa más trackear en el tiempo: los cambios de Precio de las zapatillas o los cambios en el Stock para detectar roturas?"

Atributos: "¿Hay alguna agrupación especial que no esté en las tablas (ej. Temporada Primavera/Verano) que debamos derivar de algún campo?"

RESPUESTA CLIENTE:

Muy bien, aquí ya estás entrando en decisiones de modelado importantes de verdad. Te respondo con bastante precisión porque esto impacta directamente en cómo espero la capa gold.

1) Jerarquía Producto → Variante y precios

Sí: un producto tiene múltiples variantes y el precio puede ser distinto por variante.

Esto es clave y no negociable en este modelo.

◆ Cómo funciona realmente

- **products** → nivel modelo (ej: *Nike Air Max 90*)
- **product_variants** → nivel vendible (ej: talla 42, color negro)

Y aquí lo importante:

- **list_price** está en **product_variants**
- **unit_cost** también está en variante

👉 **Esto implica:**

- **Diferentes tallas o colores pueden tener:**
 - **precios distintos**
 - **costes distintos**

♦ Implicaciones para gold

Esto es donde muchos fallan:

- ❌ No agregues a nivel producto para métricas de ventas sin cuidado
- ✅ El grano correcto es variante SIEMPRE

Porque si no:

- distorsionas precio medio
 - rompes márgenes
 - mezclas realidades distintas
-

2) Histórico (snapshots): precio vs stock

Aquí no hay un “uno u otro”. Pero sí hay prioridades claras de negocio.

● PRIORIDAD ALTA: histórico de stock

Lo más crítico para nosotros es poder analizar stock en el tiempo

Aunque tengas **inventory_stock** (foto actual), lo realmente potente es:

- **stock_movements** → histórico completo

Con esto puedes reconstruir:

- stock diario
- roturas de stock
- cobertura
- entradas/salidas

👉 Esto es CLAVE para:

- retail físico
 - reposición
 - evitar pérdida de ventas
-

● Precio: importante, pero secundario

Problema:

- No tenemos histórico de **list_price**

- Solo tenemos el valor actual en `product_variants`
-

♦ Entonces, ¿qué hacemos?

Tienes dos opciones (quiero que tomes una decisión consciente):

Opción A — No hacer snapshot de precio

- Usas precios de la venta (`sale_items.unit_final_price`)
- Más fiel a realidad transaccional

👉 Esto es lo mínimo esperado

Opción B — Snapshot de producto/variante (más avanzado)

- Trackear cambios en:
 - `list_price`
 - `unit_cost`
 - atributos

👉 Útil para:

- análisis de pricing
- evolución de catálogo

Pero:

- no es imprescindible para una primera versión de gold
-

3) Atributos derivados (muy buena pregunta)

Sí — aquí espero que aportéis valor, no solo replicar tablas.

♦ Temporada (NO viene en datos)

No existe campo explícito tipo “SS / FW”.

👉 Pero puedes derivarlo de:

- `launch_date` en `products`

Ejemplo típico:

- Primavera/Verano → meses 3–8
- Otoño/Invierno → meses 9–2

⚠ Pero esto es una aproximación, no realidad de negocio pura.

♦ Otros derivados interesantes

Te dejo algunos que sí aportan valor real:

1. Segmentación de precio

A partir de:

- `list_price`

Ejemplo:

- low / mid / premium
-

2. Edad del producto

- días desde `launch_date`

👉 Muy útil para:

- ver si un producto “se quema”
 - rotación
-

3. Cliente recurrente

No viene como flag:

- hay que calcularlo
-

4. Franja horaria de venta

A partir de `sale_ts`:

- mañana / tarde
-

5. Estado de stock derivado

A partir de `inventory_stock`:

- **en stock**
- **bajo stock**
- **rotura**

NUESTRA SOLUCION:

Hemos actualizado la capa gold para reflejar correctamente vuestro modelo de negocio: el análisis principal de producto se hace ahora a nivel variante (precio y coste por talla/color), manteniendo además la dimensión de producto a nivel modelo para navegación. También incorporamos atributos derivados de valor (season_derived, price_segment, product_age_days).

En stock, priorizamos histórico real desde movimientos (entradas/salidas) y lo complementamos con el histórico de estado desde snapshot para detectar roturas y necesidades de reposición en el tiempo. Con esto, el modelo queda preparado para análisis operativo y ejecutivo sin perder trazabilidad.

Cambios aplicados

- Jerarquía producto-variante corregida
 - Se mantiene dim_products a nivel producto (product_id) para no romper unicidad/tests.
 - Creación dim_product_variants (models/gold/dimensions/dim_product_variants.sql) como dimensión de análisis real:
 - grano product_variant_id
 - precio y coste por variante (list_price, unit_cost, gross_margin, gross_margin_pct)
 - atributos de variante (color, size_eu, variant_sku)
- Atributos derivados añadidos
 - En dim_products y/o dim_product_variants:
 - season_derived desde launch_date (SS/FW aproximado)
 - product_age_days
 - En dim_product_variants:
 - price_segment (LOW / MID / PREMIUM)
- Histórico de stock priorizado
 - Creación fct_stock_movements (models/gold/facts/fct_stock_movements.sql) desde stg_stock_movements:
 - histórico transaccional de entradas/salidas
 - movement_qty_signed, inbound_qty, outbound_qty
 - Creación daily_stock_performance (models/gold/aggregates/daily_stock_performance.sql) para análisis diario por tienda-variante.

- `date`
 - `day_of_week`
 - `is_weekend`
 - `is_sunday`
 - `is_business_day` → clave
 - `week`
 - `month`
 - `year`
-

♦ Regla de negocio

`is_business_day` = FALSE para domingos

(De momento no estamos modelando festivos reales porque no están en datos)

♦ ¿Por qué esto es importante?

Porque en reporting:

- Si no hay fila → parece “dato faltante”
- Si hay fila con 0 → es “día cerrado”

👉 Esto cambia completamente la lectura de KPIs

♦ Bonus (nivel más avanzado)

Podrías incluso cruzar con:

- `store_business_hours`

Para tener:

“tienda abierta ese día sí/no”

Pero esto ya es un plus, no obligatorio.

2) Segmentación de clientes (VIP)

Aquí te voy a exigir criterio de negocio.

👉 No tenemos definición de VIP en origen.

♦ Qué NO quiero

- ❌ inventar lógica sin explicitarla
 - ❌ meter un flag VIP “porque sí”
-

♦ Qué SÍ espero

Que propongamos una segmentación clara y defendible.

♦ Opciones razonables

Opción A — Por volumen de gasto

Ejemplo:

- VIP → clientes con `lifetime_value > X`

Problema:

- necesitas calcular LTV en gold
-

Opción B — Por frecuencia

Ejemplo:

- VIP → clientes con $\geq N$ compras

👉 alineado con:

- cliente recurrente (>1 compra)
-

Opción C — Híbrido (recomendado)

Algo tipo:

- VIP si:
 - n° compras $\geq X$
 - o gasto total $\geq Y$

♦ Mi expectativa como stakeholder

Quiero ver:

1. La lógica definida explícitamente
 2. Que sea reproducible (no hardcode raro)
 3. Que se pueda explicar en negocio
-

♦ Alternativa mejor que “VIP”

En muchos casos prefiero:

- **customer_segment:**
 - new
 - occasional
 - recurrent
 - high_value

👉 Mucho más analítico que un simple booleano

NUESTRA SOLUCION:

Hemos ajustado el modelo gold para que la operatividad se resuelva en la dimensión de fechas, no en tiendas: la dim_date ahora incluye is_business_day, is_sunday e indicadores de calendario para distinguir correctamente días cerrados de datos faltantes.

En clientes, incorporamos una segmentación reproducible y explícita (new, occasional, recurrent, high_value) junto con flags de recurrencia y VIP bajo lógica híbrida (frecuencia o gasto), evitando reglas implícitas.

Además, tratamos los tickets sin cliente identificado como WALKIN en hechos de ventas para no contaminar la segmentación de clientes registrados y mantener consistencia en BI.

Ajustes aplicados

- dim_date enriquecida (models/gold/dimensions/dim_date.sql)
 - Añadidos: week, day_name, is_weekend, is_sunday, is_business_day
 - Regla aplicada: is_business_day = FALSE en domingo
 - Se mantiene is_store_open_day como alias de compatibilidad para no romper reportes actuales
- dim_customers con segmentación explícita y reproducible (models/gold/dimensions/dim_customers.sql)

- Definición de umbrales en CTE segmentation_params (sin hardcode “oculto”)
 - Nuevas salidas:
 - is_recurrent_customer
 - customer_segment (new, occasional, recurrent, high_value)
 - is_vip (criterio híbrido: volumen o gasto)
 - is_high_value_customer
 - Se mantiene lifetime_value y total_orders
 - Tratamiento de tickets sin cliente (WALKIN)
 - Adición customer_type en fct_sales (WALKIN / IDENTIFIED)
 - Propagación de customer_type a fct_sales_tickets
 - Así la segmentación de dim_customers queda limpia para clientes identificados y los anónimos se analizan aparte sin mezclar
-
-
-

CONCLUSIONES DE CAMBIOS FINALES PARA EL CLIENTE

1. Evolución de la Capa Gold

Se ha completado una evolución orientada a **criterios de negocio**, reforzando la trazabilidad, granularidad analítica y consistencia para las herramientas de BI. El modelo actual elimina ambigüedades de definición y evita el doble conteo entre ventas y devoluciones.

2. Ventas y Transaccionalidad

- **Consolidación de fct_sales:** Establecida como tabla de hechos (*fact*) principal a nivel de línea (*sale_item_id*).
- **Desglose de Métricas:** Separación explícita de:
 - *net_sales_before_returns*: Venta neta comercial.
 - *net_sales_after_returns*: Venta neta real (post-devoluciones).
 - *gross_amount*, *discount_amount* y *amount_refunded*.
- **Segmentación de Venta:** Incorporación de *customer_type* para distinguir ventas WALKIN de ventas identificadas.
- **Soporte Analítico:** Creación de *fct_sales_tickets* (nivel *sale_id*) como tabla derivada para analizar el comportamiento de compra a nivel ticket.

3. Gestión de Devoluciones

- **Nueva Granularidad:** Incorporación de `fct_return_items` a nivel de detalle (`return_item_id`).
 - *Ventaja:* Permite el cruce directo con la venta original por `sale_item_id` sin pérdida de precisión.
- **Capa Operativa:** `fct_returns` se mantiene como tabla de cabecera (`return_id`) para asegurar la compatibilidad del reporting actual.

4. Producto y Catálogo

- **dim_product_variants:** Nueva dimensión para reflejar precio y coste a nivel de variante (`product_variant_id`). Incluye:
 - Atributos de negocio: `price_segment`, `season_derived`, `product_age_days` y márgenes.
- **dim_products:** Se mantiene como nivel maestro para navegación del modelo, incorporando derivados de calendario de lanzamiento y preservando la unicidad de clave.

5. Stock e Inventario

- **Histórico y Movimientos:**
 - `fct_stock_movements`: Base transaccional de entradas y salidas firmadas.
 - `fct_stock_status_history`: Seguimiento temporal de estados (`is_stockout`, `needs_reorder`, vigencias).
- **Análisis Operativo:** Inclusión de `daily_stock_performance`, un agregado diario por tienda-variante optimizado para tareas de reposición.

6. Dimensiones Transversales (Calendario y Clientes)

- **dim_date (Calendario):** Enriquecida con campos de operatividad (`is_weekend`, `is_sunday`, `is_business_day`). Se explicita que los domingos son no operativos para evitar huecos en los KPIs de reporting.
 - **dim_customers (Clientes):** Formalización de segmentación reproducible:
 - **Estados:** `new`, `recurrent`, `high_value`.
 - **Métricas Trazables:** `total_orders`, `lifetime_value`.
 - **Flags Analíticos:** `is_recurrent_customer`, `is_vip`, `is_high_value_customer`.
-

Resultado Final: El modelo queda preparado para análisis tanto **ejecutivo** como **operativo**, garantizando la integridad de los datos y la claridad en las métricas de rendimiento.